Trabalho — 2ª Avaliação

UNINASSAU | Disciplina: Coding | Professor: M.Sc. Tiago Martins Ribeiro

Período: 13/10 a 08/11/2026 | Entrega: 08/11 às 23:59h | Trios

> ***Resumo do trabalho:*** *o trio vai programar, em Python, um* ***jogo de tabuleiro que roda no terminal****, usando* ***matriz*** *e* `for` ***aninhado****, e vai entregar três coisas: o* ***código no GitHub****, um* ***relatório em PDF*** *e um* ***vídeo de 3 a 10 minutos****. Tudo é enviado pelo formulário até* ***08/11/2026, às 23:59h****.*

1. Tema: jogo de tabuleiro no terminal

Jogo no terminal

Desenvolvam um **jogo de tabuleiro para ser executado no terminal**, no qual o tabuleiro seja representado por uma **matriz** e percorrido com `for` **aninhado**, utilizando **apenas** o conteúdo estudado em aula (veja o item 3). O jogo deve ter, no mínimo, dois jogadores (dois humanos ou humano contra computador).

A tabela abaixo apresenta algumas sugestões e explica, de forma resumida, **como cada jogo funciona**. Vocês não precisam conhecer o jogo antes: as regras descritas aqui são suficientes para fazer o trabalho.

Jogo	Como funciona	Quem vence
Jogo da Velha (3x3)	Tabuleiro de 3 linhas e 3 colunas. Os dois jogadores se alternam, cada um marcando o seu símbolo (X ou O) em uma casa vazia.	Quem completar primeiro uma linha, uma coluna ou uma diagonal com o seu símbolo. Se o tabuleiro encher sem vencedor, é empate.
Batalha Naval simplificada (5x5 ou maior)	Cada navio ocupa algumas casas da matriz e fica escondido. A cada rodada, o jogador informa a linha e a coluna de um "tiro"; o programa responde "água" ou "acertou!" e marca a casa no tabuleiro.	Quem acertar primeiro todas as partes dos navios do adversário (ou quem precisar de menos tiros).
Campo Minado simplificado (5x5 ou maior)	Algumas casas têm minas escondidas. Os jogadores se alternam "abrindo" casas. Se a casa não tiver mina, o programa mostra nela quantas minas existem nas casas vizinhas (as até 8 casas em volta).	Quem abrir uma mina perde. Se todas as casas sem mina forem abertas, vence quem tiver aberto mais casas (ou é empate).

Jogo	Como funciona	Quem vence
Caça ao Tesouro (5x5 ou maior)	Um jogador esconde o tesouro digitando uma linha e uma coluna (depois, vários `print()` vazios "limpam" a tela para o outro não ver). O outro jogador dá palpites de posição; a cada palpite, o programa marca a casa e dá uma dica: "você está a N casas do tesouro". A distância é a diferença entre as linhas somada à diferença entre as colunas; se uma diferença der negativa, use um `if` para multiplicá-la por -1.	Os jogadores trocam de papel a cada rodada. Vence quem achar o tesouro com **menos palpites**.
Outro jogo	Outro jogo de tabuleiro de escolha do trio (por exemplo, uma versão simples de Damas ou Resta Um).	Conforme as regras do jogo, desde que respeite o item 3 e seja combinado antes com o professor.

Dica: o Jogo da Velha é a opção mais simples para começar. Batalha Naval, Campo Minado e Caça ao Tesouro exigem um pouco mais de lógica, mas continuam usando apenas o conteúdo visto em aula.

Jogos sem tabuleiro (por exemplo, Jokenpô ou adivinhação de número secreto) não atendem a esta atividade, pois não utilizam matriz. Como não é permitido usar a biblioteca `random`*, as posições secretas (minas, navios, tesouro) devem ser definidas na própria matriz, já no código, ou informadas por um dos jogadores. Em caso de dúvida sobre a viabilidade do jogo, consultem o professor antes de começar.*

2. Objetivo

Desenvolver, em trio, um **programa funcional de jogo de tabuleiro** que utilize de forma integrada os conteúdos estudados na disciplina, com destaque para **matrizes e `for` aninhado**, documentando o processo com **boas práticas de versionamento (GitHub)**, **relatório técnico** e **vídeo explicativo**.

3. Restrição de conteúdo (muito importante)

O programa deve ser desenvolvido **estritamente** com os conteúdos já estudados:

Permitido	Proibido (ainda não estudado)
Tipos de dados e variáveis	Funções criadas pelo aluno (`def`, `lambda`)
Operadores (aritméticos, relacionais e lógicos)	Dicionários, tuplas e conjuntos
Estruturas de decisão (`if`, `elif`, `else`)	Vetores (listas de uma dimensão) fora da matriz
Estruturas de repetição (`while`, `for` e `range()`)	Métodos de lista e de texto (como `append`, `isdigit`, `upper`) e list comprehension

Permitido	**Proibido (ainda não estudado)**
Matrizes (lista de listas) e `for` aninhado	Bibliotecas (`import`, como `random` e `time`) e `try/except`
Funções nativas básicas: `print()`, `input()`, `int()`, `float()` e `str()`	Classes e orientação a objetos

Sobre a matriz: o tabuleiro deve ser declarado por extenso, linha a linha (por exemplo, três listas de três posições para um tabuleiro 3x3), e acessado por `tabuleiro[i][j]`, em que `i` é a linha e `j` é a coluna.

> ***Atenção:*** *trabalhos que utilizem qualquer um dos recursos proibidos terão desconto na nota de "Código funcional" (item 7), pois não aplicam corretamente os conteúdos permitidos.*

4. Requisitos de conteúdo (obrigatório usar todos)

Conteúdo	**Onde deve aparecer no programa**
Tipos de dados / variáveis	Pontuações, símbolos ou peças dos jogadores, contadores de rodadas e de partidas, jogada lida do teclado e indicador de quem joga (inteiros, textos e booleanos)
Operadores (aritméticos, relacionais e lógicos)	Cálculo do placar e de porcentagens; comparação entre casas da matriz; checagem dos limites do tabuleiro e combinação de condições com `and`, `or` e `not`
Estruturas de decisão (`if`, `elif`, `else`)	Definição do vencedor ou empate, validação das jogadas (casa livre ou ocupada, dentro ou fora do tabuleiro) e tratamento das regras especiais do jogo
Estruturas de repetição (`while`)	Menu principal, repetição da leitura até a entrada ser válida, sequência de jogadas da partida e opção de jogar novamente
Matrizes e `for` aninhado	Tabuleiro do jogo: exibição, reinício, consulta e atualização das casas, verificação de linhas, colunas e diagonais

5. Funcionalidades e desafios

a) Funcionalidades obrigatórias

Adaptem as funcionalidades abaixo ao jogo escolhido:

- **Menu interativo** no terminal, controlado por `while`, com opções (por exemplo: jogar, ver regras, ver placar, sair);

- **Tabuleiro em matriz**, exibido a cada jogada com `for` aninhado e com os números das linhas e colunas visíveis para o jogador;
- **Leitura das jogadas com validação:** se o jogador digitar algo errado (letra, símbolo, campo vazio, número fora do tabuleiro ou casa já ocupada), o programa **não pode fechar com erro**; ele deve mostrar uma mensagem e pedir a jogada de novo (veja a explicação 1, abaixo);
- **Verificação automática de vitória e empate usando laços** (`for`): o próprio programa percorre o tabuleiro para descobrir se alguém venceu ou se deu empate (veja a explicação 2, abaixo);
- **Placar acumulado** entre as partidas (vitórias de cada jogador e empates), exibido ao final de cada partida;
- **Opção de jogar novamente** sem reiniciar o programa, com a matriz sendo "limpa" por `for` aninhado;
- **Mensagens claras** ao jogador: de quem é a vez, regras do jogo e resultado da partida.

Explicação 1 — O que é "validar a entrada" (o programa nunca pode quebrar)?

Quando o programa espera um número e o jogador digita uma letra, o comando `int("a")` gera um erro (`ValueError`) e o Python **encerra o jogo na hora**. Validar a entrada significa **conferir o que foi digitado antes de converter** e, se estiver errado, avisar o jogador e pedir de novo, usando `while`.

Como não podemos usar `try`/`except` nem `isdigit()`, uma forma simples é **comparar o texto digitado com cada opção válida**, usando `for` e `str()`:

```
valida = False
while valida == False:
    entrada = input("Escolha a coluna (1 a 7): ")   # entrada é texto
    for n in range(1, 8):            # opções válidas: "1", "2", ..., "7"
        if entrada == str(n):
            valida = True
            coluna = n - 1           # índice da matriz (0 a 6)
    if valida == False:
        print("Entrada inválida! Digite um número de 1 a 7.")
```

Com isso, letras, símbolos, campo vazio e números fora do intervalo são recusados sem quebrar o programa. Depois, com um `if`, confiram se a casa escolhida está livre; se estiver ocupada, peçam outra jogada.

Explicação 2 — O que é "verificar a vitória com laços"?

Para saber se alguém venceu, o programa precisa olhar as linhas, as colunas e as diagonais do tabuleiro. Existem duas formas de fazer isso:

Forma NÃO aceita — escrever um `if` para cada combinação, uma por uma. No Jogo da Velha isso dá 8 testes quase iguais, copiados e colados. O código fica enorme, repetitivo e fácil de errar:

```
# NÃO aceito: cada linha, coluna e diagonal escrita à mão
if tab[0][0] == "X" and tab[0][1] == "X" and tab[0][2] == "X":
    venceu = True
if tab[1][0] == "X" and tab[1][1] == "X" and tab[1][2] == "X":
    venceu = True
# ... e mais 6 testes iguais, só trocando os números
```

Forma aceita — usar `for` para repetir o mesmo teste automaticamente. O laço troca o número da linha sozinho, então um único `if` serve para todas as linhas:

```
# ACEITO: o for repete o mesmo teste para cada linha i (0, 1 e 2)
for i in range(3):
    if tab[i][0] == simbolo and tab[i][1] == simbolo and tab[i][2] == simbolo:
        venceu = True
```

Façam o mesmo para as colunas (trocando de lugar o índice que varia) e para as duas diagonais. Para o **empate**, usem `for` aninhado para contar as casas vazias: se não sobrou nenhuma e ninguém venceu, a partida terminou empatada.

```
# for aninhado: conta quantas casas ainda estão vazias
vazias = 0
for i in range(3):
    for j in range(3):
        if tab[i][j] == " ":
            vazias = vazias + 1
if vazias == 0 and venceu == False:
    print("Deu empate!")
```

Em jogos como Batalha Naval, Campo Minado ou Caça ao Tesouro a ideia é a mesma: usar `for` aninhado para percorrer a matriz e **contar** (por exemplo, quantas partes de navio ainda restam ou quantas casas sem mina faltam abrir), em vez de testar casa por casa à mão.

> ***Observação:*** *os trechos acima são apenas exemplos para entender a ideia. O trio deve adaptá-los ao seu jogo e explicar, no relatório e no vídeo, como a lógica funciona.*

b) Desafios de lógica (escolher no mínimo 1)

Para elevar o nível do trabalho, o trio deve implementar **pelo menos um** dos desafios abaixo, sempre usando apenas o conteúdo permitido. A coluna “Nível” ajuda a escolher de acordo com a experiência do grupo.

Desafio	O que fazer	Nível
Campeonato “melhor de N”	No menu, o jogador escolhe uma série “melhor de 3” ou “melhor de 5”. O programa conta as vitórias e encerra a série assim que alguém garante a maioria (por exemplo, 2 vitórias em uma melhor de 3), anunciando o campeão.	Básico
Estatísticas da sessão	Ao escolher “Sair”, mostrar: total de partidas, vitórias de cada jogador, empates, a **porcentagem** de vitórias de cada um (número real, ex.: 66.7%) e a maior sequência de vitórias seguidas.	Básico

Desafio	O que fazer	Nível
Casas especiais	Algumas casas da matriz recebem um símbolo especial (ex.: `#` bloqueada, `B` bônus, `A` armadilha). Quando o jogador escolhe uma delas, algo muda no jogo: perde a vez, joga de novo ou ganha pontos.	Intermediário
Tabuleiro ampliado	Usar um tabuleiro maior (no mínimo 5x5, ou mais) em que a vitória exige uma sequência de K peças (ex.: 4 em linha), verificada na horizontal, na vertical e nas duas diagonais usando laços.	Intermediário
Computador como adversário	O computador joga sozinho contra o humano seguindo regras fixas, nesta ordem: 1) se puder vencer nesta jogada, vence; 2) senão, se o humano estiver prestes a vencer, bloqueia; 3) senão, ocupa a primeira casa livre. Não pode haver sorteio (`random` não é permitido).	Avançado

O relatório e o vídeo devem informar **qual(is) desafio(s) foi(ram) escolhido(s)** e **em que parte do código** cada um aparece. Se o jogo escolhido não comportar algum desafio, consultem o professor para combinar uma adaptação.

6. Entregáveis

a) Código no GitHub

- Repositório **público** (ou privado, com o professor adicionado como colaborador @profmartinz).
- **Devem existir *commits* dos TRÊS integrantes do trio**, distribuídos ao longo do período, com **no mínimo 5 *commits* por integrante, em pelo menos 3 dias diferentes**. Não é aceito que apenas um aluno programe e os demais apareçam só no README. Dividam as tarefas (por exemplo: aluno A desenvolve a matriz, a exibição do tabuleiro e a validação de entradas; aluno B, as regras e a verificação de vitória por laços; aluno C, o placar, o reinício da partida e o desafio) para gerar histórico real de cada integrante.
- **Não serão considerados válidos** *commits* feitos por apenas um aluno no mesmo dia ou em dias próximos à entrega da avaliação; nesses casos haverá desconto de pontos.
- As mensagens dos *commits* devem ser descritivas (por exemplo: “Adiciona validação da coluna escolhida”); mensagens genéricas, como “teste” ou “ajustes”, reduzem a nota deste item.
- **README.md** explicando como executar o programa, as regras do jogo, o(s) desafio(s) escolhido(s) e quem fez cada parte.

b) Relatório técnico

- **Baseado no *template* do professor Martins**, disponível no site do professor (https://profmartins.com.br/computacao/). A estrutura do relatório e os tópicos estão descritos no *template*; leiam com atenção.

- Deve explicar, para cada conteúdo da tabela do item 4, **qual função ele cumpre no código** (exemplo: "o `for` aninhado foi utilizado para percorrer cada linha e cada coluna da matriz do tabuleiro, permitindo verificar se havia três peças iguais em sequência...").
- Deve apresentar o **teste de mesa** de um trecho com `for` aninhado (tabela com os valores de `i`, `j` e da casa consultada a cada passo).
- Deve conter uma **tabela de testes** com no mínimo 8 casos (entrada, resultado esperado e resultado obtido), incluindo pelo menos 4 entradas inválidas (letra, campo vazio, número fora do intervalo e casa já ocupada).
- Deve indicar **qual(is) desafio(s) de lógica** (item 5.b) foi(ram) implementado(s) e em que parte do código.
- Deve conter o **link do repositório GitHub** e o **link do vídeo**.
- Deve identificar os integrantes do trio e a divisão de tarefas entre eles.
- Entrega em PDF.

c) Vídeo explicativo (YouTube, não listado)

O trio deverá gravar um vídeo com os seguintes requisitos:

- **Duração:** mínimo de 3 minutos e máximo de 10 minutos.
- **Estrutura sugerida:**
  a. *Demonstração (primeira parte):* o jogo rodando, com **uma partida completa até a vitória**, **uma partida terminando em empate** (ou situação equivalente), **pelo menos três entradas inválidas** sendo tratadas e o(s) **desafio(s) escolhido(s)** em funcionamento;
  b. *Explicação técnica (segunda parte):* a lógica do código. Não é necessário ler linha a linha, mas é **obrigatório explicar a matriz e os `for` aninhados** (exibição e verificação de vitória) e como cada estrutura funciona no jogo.
- **Todos os três integrantes devem falar** no vídeo (não é necessário aparecer na câmera; a voz é suficiente).
- É **obrigatório mostrar a tela do computador** durante a explicação, apresentando o código e seu funcionamento na prática.
- Vídeos em que o aluno apenas grava o próprio rosto, sem apresentar a tela, o código e o funcionamento, **serão zerados automaticamente**.
- Publicar no YouTube como **"não listado"** e incluir o link no relatório.

d) Formulário de entrega

- A entrega deve ser feita pelo formulário: **https://forms.gle/rT4c2MRCjb2xkiga7**
- Devem ser informados: **nomes dos três integrantes**, **link do GitHub**, **link do vídeo** e **upload do arquivo do relatório**.
- **Prazo: até 08/11/2026 (domingo), às 23:59h.** Trabalhos entregues após esse dia e horário receberão **nota 0 (zero)** na avaliação.

7. Critérios de avaliação (10,0 pontos)

O critério **Código funcional** vale 4,0 pontos e é dividido em seis partes. A coluna **"O que será observado"** mostra exatamente o que o professor vai conferir em cada item.

Critério	O que será observado	Pontos
Código funcional: tabuleiro em matriz	O tabuleiro é uma matriz (lista de listas) e aparece na tela a cada jogada, desenhado com `for` aninhado, com os números das linhas e colunas visíveis.	0,6
Código funcional: vitória e empate	O programa descobre sozinho quem venceu ou se houve empate, usando laços (`for`) para percorrer linhas, colunas e diagonais, como na Explicação 2 do item 5. Se cada combinação for testada com um `if` separado, escrito à mão, este item não é atendido.	0,8
Código funcional: validação das entradas	Ao digitar letra, símbolo, campo vazio, número fora do tabuleiro ou casa ocupada, o jogo mostra uma mensagem e pede de novo, **sem fechar com erro**, como na Explicação 1 do item 5.	0,6
Código funcional: menu, placar e reinício	Menu controlado por `while`; placar acumulado entre as partidas; opção de jogar novamente com o tabuleiro limpo, sem reiniciar o programa.	0,6
Código funcional: desafio de lógica	Pelo menos **um** desafio do item 5.b funcionando corretamente e apresentado no relatório e no vídeo.	1,0
Código funcional: organização e restrições	Nomes de variáveis claros, comentários explicando os laços e uso apenas do conteúdo permitido no item 3.	0,4
GitHub	*Commits* dos três integrantes (mínimo de 5 por integrante, em pelo menos 3 dias diferentes), mensagens descritivas e README.md completo.	2,0
Relatório	Segue o *template*; explica a função de cada conteúdo no código; apresenta o teste de mesa, a tabela de testes, o(s) desafio(s) e os links.	2,0
Vídeo	Entre 3 e 10 minutos; demonstração do jogo (vitória, empate, entradas inválidas e desafio); explicação técnica mostrando a tela; os três integrantes falam.	2,0
Total		**10,0**

Observação: O trabalho será considerado na composição da nota da 2ª avaliação, juntamente com a avaliação presencial, que permanecerá mantida.

A composição da nota será realizada da seguinte forma: o trabalho terá valor de 10,0 pontos, e a avaliação presencial também terá valor de 10,0 pontos. A nota final da 2ª avaliação será obtida por meio da média aritmética das duas notas.

O aluno que não realizar o trabalho receberá nota 0,0 nessa atividade. Nesse caso, a nota final da 2ª avaliação será calculada considerando a nota obtida na avaliação presencial e a nota zero atribuída ao trabalho.

8. Cronograma sugerido (13/10 a 08/11)

Como o trabalho é realizado de forma assíncrona, segue uma sugestão de ritmo para orientar o trio:

Período	Atividade
13/10 a 19/10	Escolha do jogo, divisão de tarefas no trio, estrutura inicial do projeto no GitHub e desenho do tabuleiro no papel
20/10 a 27/10	Matriz do tabuleiro, exibição com `for` aninhado, menu e leitura com validação das jogadas
28/10 a 01/11	Regras do jogo, verificação de vitória e empate por laços, placar, reinício da partida e desafio de lógica
02/11 a 04/11	Testes (incluindo entradas inválidas), ajustes finais e escrita do relatório
05/11 a 07/11	Gravação e edição do vídeo (colocar o link do vídeo no relatório)
08/11	Envio final pelo formulário (até 23:59h)

9. Orientações finais

- Comecem cedo e façam *commits* pequenos e frequentes, cada integrante em seu próprio computador/conta.
- Antes de programar, **desenhem o tabuleiro no papel** e façam o teste de mesa dos laços: isso evita os erros mais comuns de matriz (linha e coluna trocadas, índice fora dos limites).
- Testem o jogo com entradas inesperadas (letras no lugar de números, campo vazio, números fora do intervalo, casa já ocupada) para evitar erros durante a demonstração.
- Comentem o código, principalmente os `for` aninhados, explicando o que cada laço percorre.
- Confiram, antes de enviar, se o vídeo está como “não listado” e se o repositório está acessível ao professor.
- Dúvidas devem ser esclarecidas com o professor com antecedência; não deixem para os últimos dias.

Bom trabalho!

Prof. M.Sc. Tiago Martins Ribeiro — UNINASSAU